

Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Maitry Banduke	EN22CS301571
2	Mohd Quasim	EN22CS301603
3	Mrunali Kamerikar	EN22CS301618
4	Muskan Asija	EN22CS301623

Group Name: GROUP 03D6

Project Number: DO-14

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2026

Table of contents

Sr. No.	Section / Chapter	Page No.
1	Problem Statement & Objectives	3-4
1.1	Problem Statement	3
1.2	Project Objectives	3
1.3	Scope of the Project	4
2	Proposed Solution	4-11
2.1	Key Features	4-5
2.2	Overall Architecture / Workflow	5-10
2.2.1	DFD (Data Flow Diagram)	6-7
2.2.2	Use Case Diagram	7
2.2.3	Class Diagram	8
2.2.4	Activity Diagram	9-10
2.3	Tools & Technologies Used	11
3	Results & Output	11-15
3.1	Outputs (Screenshots)	11-13
3.2	Reports (Dashboards / Models)	14
3.3	Key Outcomes	14
	Conclusion	15
	Future Scope & Enhancements	15

Chapter 1: Problem Statement & Objectives

1.1 Problem Statement

In modern web application development, maintaining consistency across development, testing, and production environments is a major challenge. Traditional deployment methods often lead to issues such as dependency conflicts, environment mismatches, and scalability limitations. These challenges become more prominent when multiple developers collaborate on the same project or when applications are deployed across different systems.

The Entertainment React Service, being a frontend-based application, requires efficient handling of dependencies, optimized build processes, and seamless integration with backend services and databases. Without proper containerization, the application may face inconsistencies in performance, increased deployment time, and difficulty in scaling.

Additionally, managing the application lifecycle manually can result in inefficiencies, lack of automation, and increased chances of human error. There is a need for a standardized, automated, and scalable solution that ensures smooth development, testing, and deployment processes.

1.2 Project Objectives

The primary objectives of the Dockerized Entertainment React Service project are:

- To **containerize the React-based entertainment application** using Docker for consistent execution across environments.
- To design and implement **multi-stage Docker builds** to reduce image size and improve security.
- To ensure **environment consistency** between development, testing, and production systems.
- To create a **Docker Compose configuration** for managing application services and dependencies efficiently.
- To implement **CI/CD pipelines** using tools like Jenkins or GitHub Actions for automated build, testing, and deployment.
- To enable **scalability and orchestration** of the application using Kubernetes.
- To improve **deployment speed and reliability** by automating the software delivery process.
- To follow **DevOps best practices** for continuous integration, continuous delivery, and infrastructure as code.

1.3 Scope of the Project

The scope of this project includes the design, development, containerization, and deployment of a React-based entertainment service using modern DevOps practices.

In Scope:

- Development of a React-based frontend application.
- Containerization using Docker with optimized multi-stage Dockerfiles.
- Service orchestration using Docker Compose.
- Integration of CI/CD pipelines using Jenkins or GitHub Actions.
- Deployment and scaling using Kubernetes.
- Version control and collaboration using Git.
- Ensuring portability across different environments.

Out of Scope:

- Development of complex backend microservices (only basic integration considered).
- Production-grade cloud infrastructure cost optimization.
- Security auditing beyond basic Docker and CI/CD practices.

Chapter 2: Proposed Solution

2.1 Key Features

The Dockerized Entertainment React Service is designed with modern DevOps principles to ensure efficiency, scalability, and consistency. The key features of the system include:

- **Containerization using Docker:**
The application is packaged into containers, ensuring consistency across all environments.
- **Multi-Stage Docker Builds:**
Optimized Dockerfiles are used to reduce image size and improve security by separating build and runtime environments.
- **Service Orchestration with Docker Compose:**
Multiple services such as frontend and database are managed together using a single configuration file.
- **CI/CD Integration:**
Automated pipelines using Jenkins or GitHub Actions enable continuous integration and continuous deployment.

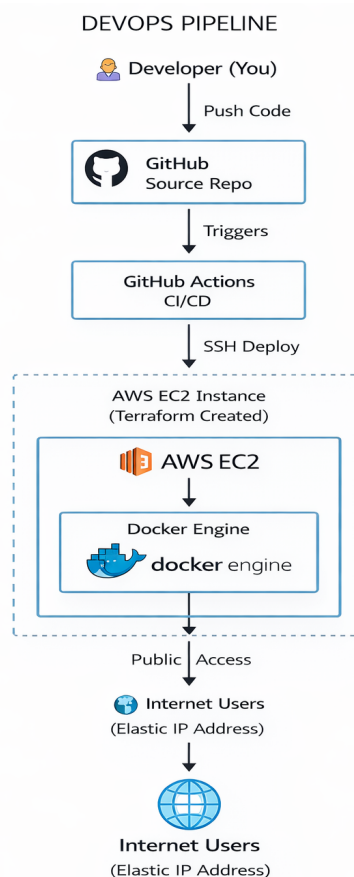
- **Scalability with Kubernetes:**
The application can be scaled horizontally using Kubernetes for handling increased user load.
- **Version Control with Git:**
Efficient source code management and collaboration among team members.
- **Environment Consistency:**
Eliminates "works on my machine" issues by standardizing the runtime environment.

2.2 Overall Architecture / Workflow

The system follows a modular and containerized architecture where each component is independently managed but works together seamlessly.

Workflow Overview:

1. Developer writes code and pushes it to the Git repository.
2. CI/CD pipeline is triggered automatically.
3. Application is built using Docker multi-stage build.
4. Docker images are created and stored in a registry.
5. Docker Compose or Kubernetes deploys the application.
6. Users access the application through a web interface.



2.2.1 DFD (Data Flow Diagram)

The Data Flow Diagram represents how data moves through the system.

Description:

- Users interact with the React frontend.
- The frontend sends requests to backend APIs (if integrated).
- Backend communicates with the database.
- Responses are sent back to the frontend and displayed to users.

Key Components:

- External Entity: User
- Process: React Application
- Data Store: Database
- Data Flow: API Requests/Responses

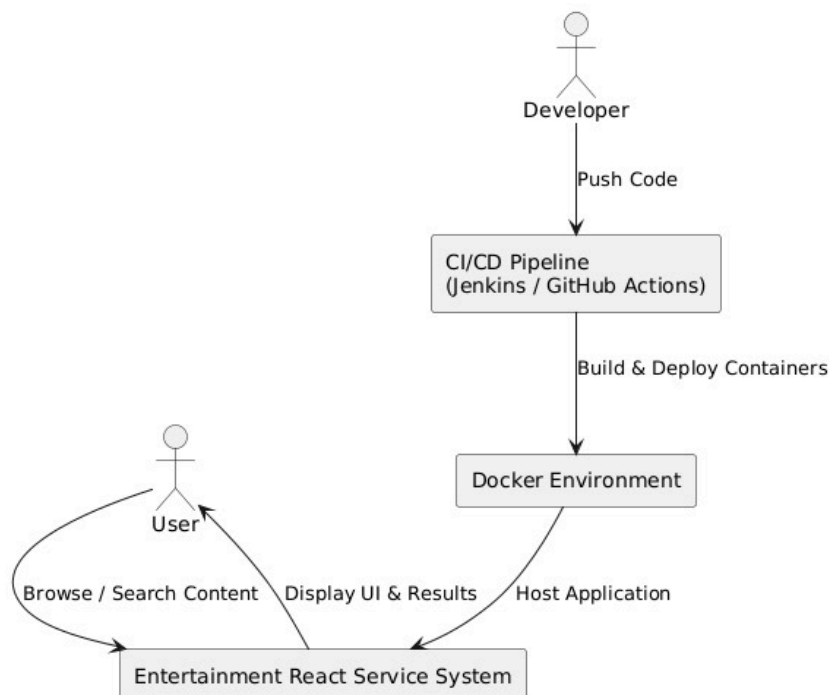


Figure 1 : DFD Level 0

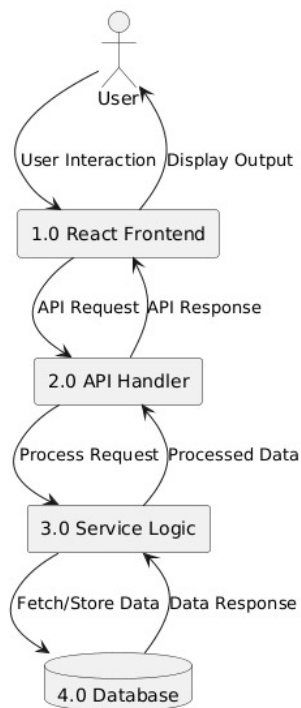


Figure 2 : DFD Level 1

2.2.2 Use Case Diagram

The Use Case Diagram illustrates the interaction between users and the system.

Actors:

- User (End-user)
- Admin (optional for management)

Use Cases:

- View entertainment content
- Search content
- Interact with UI
- Admin manages content

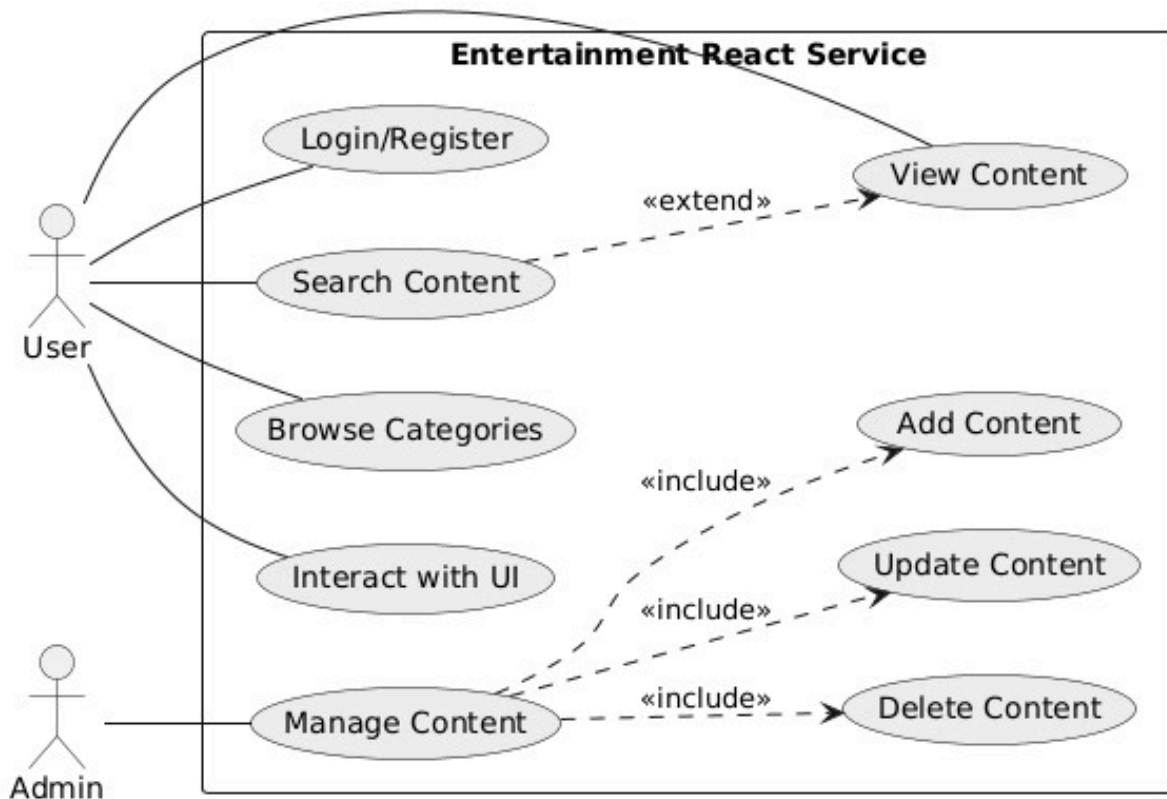


Figure 3 : Use Case Diagram

2.2.3 Class Diagram

The Class Diagram represents the structure of the system in terms of classes and their relationships.

Main Classes:

- User Interface Component
- Service Handler
- API Handler
- Data Model

Relationships:

- UI interacts with services
- Services communicate with APIs
- APIs interact with the database

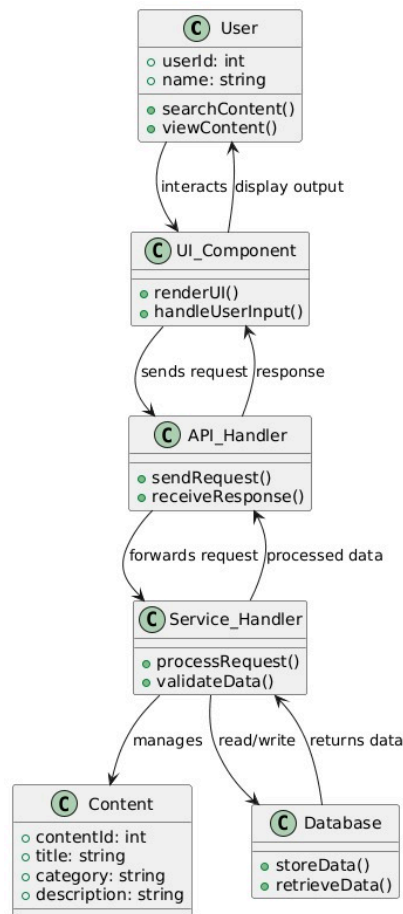


Figure 4 : Class Diagram

2.2.4 Activity Diagram

The Activity Diagram shows the flow of actions within the system.

Flow:

1. User opens the application
2. UI loads data
3. User interacts (search/view)
4. Request sent to backend/API
5. Data retrieved from database
6. Response displayed to user

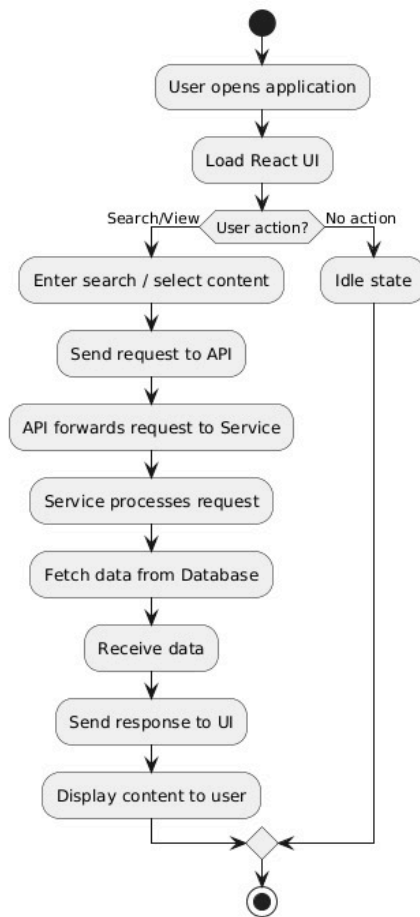


Figure 5 : Activity Diagram

2.3 Tools & Technologies Used

The project utilizes a set of modern tools and technologies:

Frontend:

- **React.js:** For building interactive user interfaces

Backend:

- **Python / FastAPI or Flask:** For handling APIs

DevOps Tools:

- **Docker:** Containerization of the application
- **Docker Compose:** Multi-container orchestration
- **Kubernetes:** Container orchestration and scaling

- **GitHub Actions:** CI/CD automation

Version Control:

- **Git & GitHub:** Source code management and collaboration

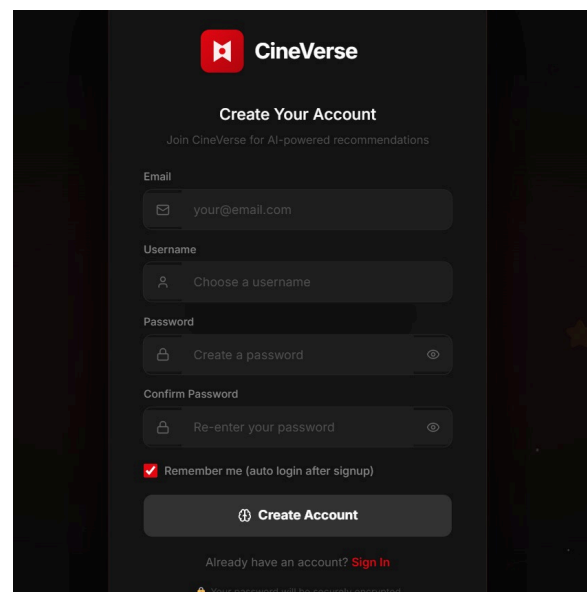
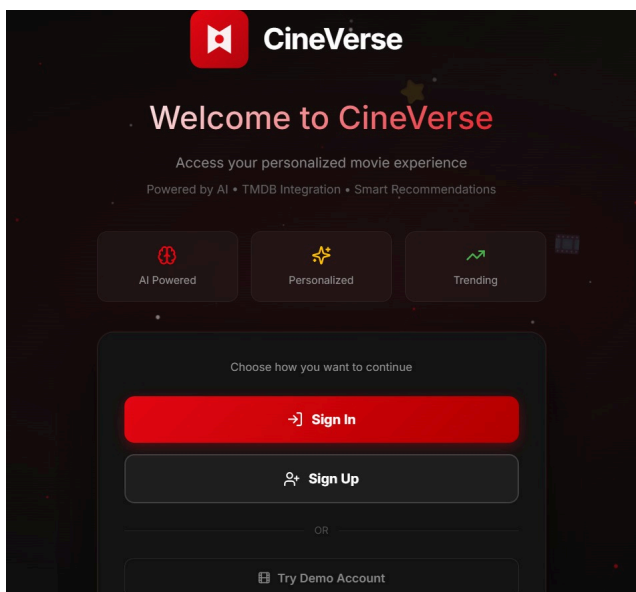
Chapter 3: Results & Output

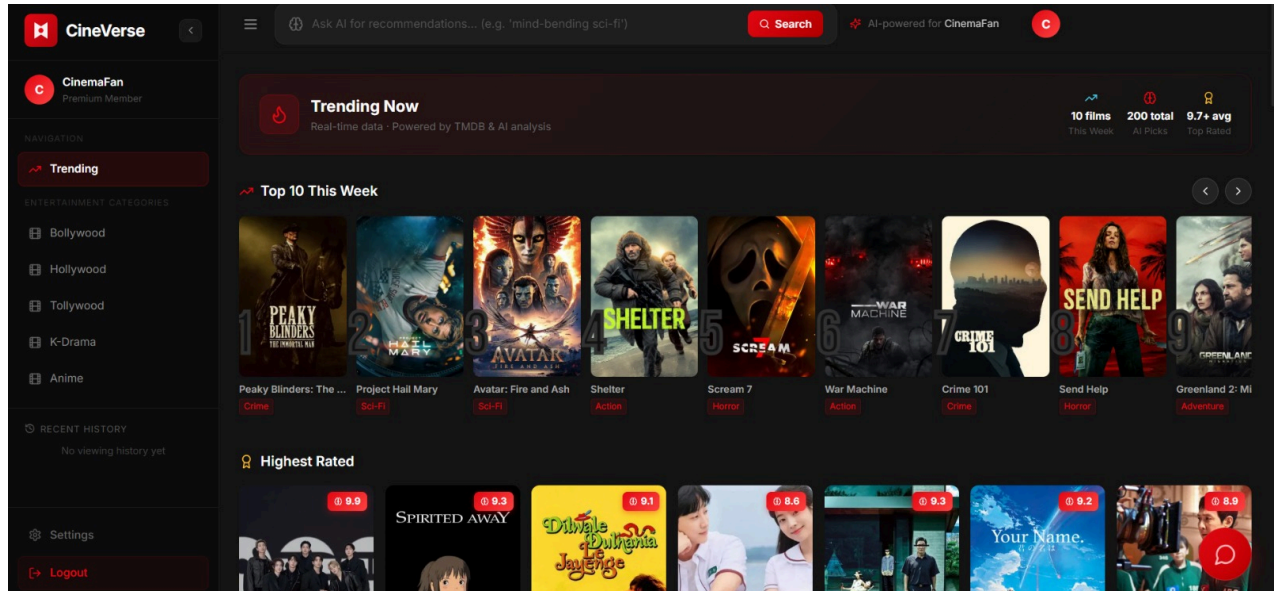
3.1 Outputs (Screenshots)

The implementation of the Dockerized Entertainment React Service produced several important outputs demonstrating the successful setup and execution of the system.

1. Application Interface

- The React-based frontend successfully renders the entertainment platform UI.
- Users can interact with the interface to view and explore content.
- The UI is responsive and works across different devices and screen sizes.





2. Docker Container Execution

- Containers are built and run successfully using Docker.
- The application runs inside isolated environments without dependency conflicts.
- Docker commands such as docker build and docker run execute without errors.

```

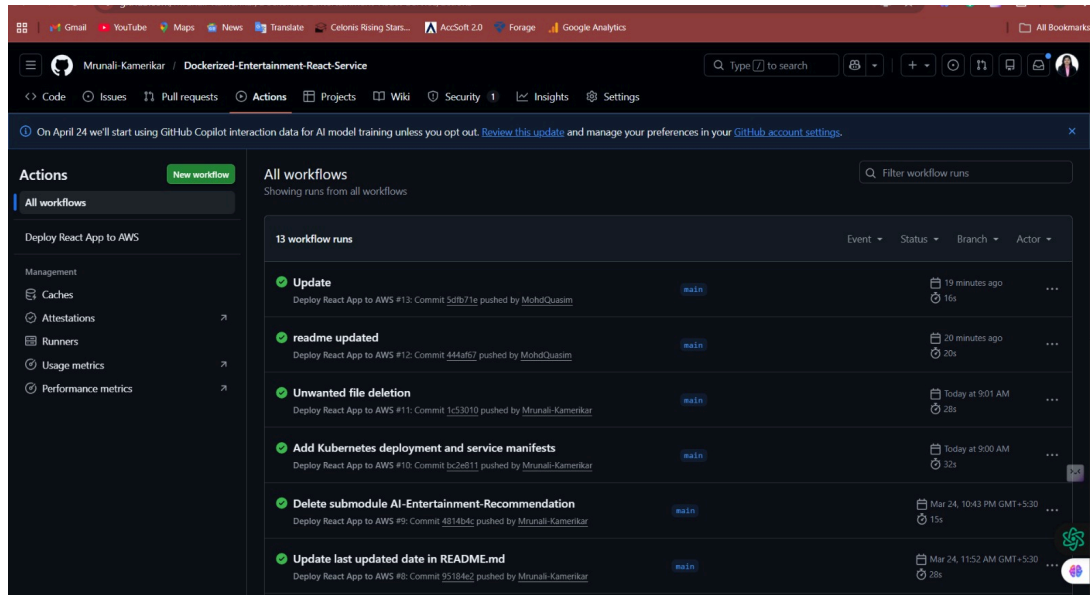
ubuntu@ip-172-31-36-154:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
a351addccb01   ai-entertainment-devops_entertainment-app   "/docker-entrypoint..." 13 minutes ago Up 13 minutes 0.0.0.0:3000->80/tcp, [::]:3000->80/tcp   entertai
ent-react
ubuntu@ip-172-31-36-154:~$ docker logs entertainment-react
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/26 05:11:15 [notice] 1#1: using the "epoll" event method
2026/03/26 05:11:15 [notice] 1#1: nginx/1.29.6
2026/03/26 05:11:15 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/03/26 05:11:15 [notice] 1#1: OS: Linux 6.9.0-1050-aws
2026/03/26 05:11:15 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/26 05:11:15 [notice] 1#1: start worker processes
2026/03/26 05:11:15 [notice] 1#1: start worker process 30
2026/03/26 05:11:15 [notice] 1#1: start worker process 31
ubuntu@ip-172-31-36-154:~$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
a351addccb01   ai-entertainment-devops_entertainment-app   "/docker-entrypoint..." 17 minutes ago Up 17 minutes 0.0.0.0:3000->80/tcp, [::]:3000->80/tcp   entertai
ent-react
ubuntu@ip-172-31-36-154:~$
  
```

3. Docker Compose Execution

- Multiple services (frontend and database) are orchestrated using Docker Compose.
- The docker-compose up command launches all services simultaneously.
- Services communicate efficiently within the Docker network.

4. CI/CD Pipeline Execution

- Automated pipelines are triggered on code commits.
- Build, test, and deployment stages are executed successfully.
- Logs confirm successful pipeline execution without failures.



5. Kubernetes Deployment

- Application is deployed on a Kubernetes cluster.
- Pods, services, and deployments are created and managed effectively.
- Application scaling is demonstrated by increasing replica count.

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service> kubectl get nodes
NAME STATUS ROLES AGE VERSION
minikube Ready control-plane 25h v1.35.1
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service> kubectl get deployments
NAME READY UP-TO-DATE AVAILABLE AGE
entertainment-deployment 2/2 2 2 24h
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service> kubectl get pods
NAME READY STATUS RESTARTS AGE
entertainment-deployment-6bf5dff9c7-mdzz5 1/1 Running 1 (4m23s ago) 24h
entertainment-deployment-6bf5dff9c7-qz6bg 1/1 Running 1 (4m23s ago) 24h
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service> kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
entertainment-service NodePort 10.98.88.226 <none> 80:30007/TCP 25h
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 25h
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service> kubectl get all
NAME READY STATUS RESTARTS AGE
pod/entertainment-deployment-6bf5dff9c7-mdzz5 1/1 Running 1 (4m36s ago) 24h
pod/entertainment-deployment-6bf5dff9c7-qz6bg 1/1 Running 1 (4m36s ago) 24h

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/entertainment-service NodePort 10.98.88.226 <none> 80:30007/TCP 25h
service/kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 25h

NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/entertainment-deployment 2/2 2 2 24h

NAME DESIRED CURRENT READY AGE
replicaset.apps/entertainment-deployment-6bf5dff9c7 2 2 2 24h
PS C:\Users\ASUS\OneDrive\Documents\GitHub\Dockerized-Entertainment-React-Service>

```

3.2 Reports (Dashboards / Models)

The project also includes monitoring and reporting mechanisms to track system performance and pipeline activities.

1. CI/CD Dashboard

- Displays build status, success/failure rates, and execution time.
- Helps in identifying issues quickly during development.

2. Container Monitoring

- Shows running containers, resource usage (CPU, memory), and uptime.
- Ensures optimal performance of the application.

3. Kubernetes Dashboard

- Visualizes cluster status, nodes, pods, and deployments.
- Helps in managing and scaling the application efficiently.

4. Logs and Reports

- Detailed logs are generated for builds, deployments, and runtime execution.
- Useful for debugging and performance analysis.

3.3 Key Outcomes

The project successfully achieved its intended objectives and demonstrated the effectiveness of DevOps practices.

- **Consistency Across Environments:**
The application runs uniformly across development, testing, and production environments.
- **Improved Deployment Speed:**
Automated pipelines significantly reduce manual effort and deployment time.
- **Scalability:**
Kubernetes enables easy scaling of the application based on demand.
- **Reduced Errors:**
Automation minimizes human errors in the deployment process.
- **Optimized Performance:**
Multi-stage Docker builds reduce image size and improve efficiency.

Conclusion

The Dockerized Entertainment React Service demonstrates how modern DevOps practices can streamline application development and deployment. By integrating containerization, CI/CD pipelines, and orchestration tools, the project ensures a reliable, scalable, and efficient system. The use of Docker and Kubernetes significantly improves portability and scalability, making the application ready for real-world deployment scenarios.

Future Scope & Enhancements

The project can be further enhanced with the following improvements:

- **Advanced Monitoring Tools:**
Integration with tools like Prometheus and Grafana for real-time monitoring.
- **Cloud Deployment:**
Deploying the application on cloud platforms such as AWS, Azure, or Google Cloud.
- **Microservices Architecture:**
Expanding the system into fully independent microservices.
- **Security Enhancements:**
Implementing advanced security practices like container scanning and secrets management.
- **AI-Based Recommendations:**
Adding intelligent recommendation systems for personalized user experience.
- **Auto-Scaling:**
Implementing automatic scaling based on traffic using Kubernetes.